

GEMINI

```
import math
```

```
# --- FUNÇÕES DE PROCESSAMENTO (LÓGICA) ---
```

```
def validar_e_filtrar_medicoes(medicoes_brutas, faixa_min=0.0, faixa_max=500.0):
```

```
    """
```

```
    1 e 2. Recebe a lista bruta e valida cada medição.
```

```
    Descarta valores não numéricos ou fora da faixa aceitável.
```

```
    """
```

```
    validas = []
```

```
    descartadas = 0
```

```
    for item in medicoes_brutas:
```

```
        try:
```

```
            valor = float(item)
```

```
            if faixa_min <= valor <= faixa_max:
```

```
                # 3. Conversão para unidade de engenharia (Volts) já é direta
```

```
                validas.append(valor)
```

```
            else:
```

```
                descartadas += 1
```

```
        except (ValueError, TypeError):
```

```
            descartadas += 1
```

```
    return validas, descartadas
```

```
def calcular_estatisticas(medicoes_validas):
```

```
    """
```

```
    4. Calcula média, mínimo, máximo e desvio padrão amostral das medições válidas.
```

```
    """
```

```
    if not medicoes_validas:
```

```
        return {}
```

```
    qtd = len(medicoes_validas)
```

```
    minimo = min(medicoes_validas)
```

```
    maximo = max(medicoes_validas)
```

```
    media = sum(medicoes_validas) / qtd
```

```
    if qtd > 1:
```

```
        variancia = sum((x - media) ** 2 for x in medicoes_validas) / (qtd - 1)
```

```
        desvio_padrao = math.sqrt(variancia)
```

```
    else:
```

```
        desvio_padrao = 0.0
```

```
    return {
```

```

    "minimo": minimo,
    "maximo": maximo,
    "media": media,
    "desvio_padrao": desvio_padrao
}

```

```

def classificar_medicao(valor, nominal, limiar_alerta, limiar_critico):

```

```

    """

```

```

    5. Classifica uma única medição com base nos desvios percentuais do nominal.

```

```

    """

```

```

    desvio_percentual = abs(valor - nominal) / nominal

```

```

    if desvio_percentual >= limiar_critico:

```

```

        return "Crítico"

```

```

    elif desvio_percentual >= limiar_alerta:

```

```

        return "Alerta"

```

```

    return "Normal"

```

```

def analisar_estados_e_sequencias(medicoes_validas, nominal, limiar_alerta=0.10,
limiar_critico=0.20):

```

```

    """

```

```

    5 e 6. Classifica os estados, conta os críticos e identifica a maior
    sequência consecutiva de medições acima do limiar crítico.

```

```

    """

```

```

    contagem_estados = {"Normal": 0, "Alerta": 0, "Crítico": 0}

```

```

    maior_seq_critica = 0

```

```

    seq_critica_atual = 0

```

```

    for valor in medicoes_validas:

```

```

        estado = classificar_medicao(valor, nominal, limiar_alerta, limiar_critico)

```

```

        contagem_estados[estado] += 1

```

```

        if estado == "Crítico":

```

```

            seq_critica_atual += 1

```

```

            if seq_critica_atual > maior_seq_critica:

```

```

                maior_seq_critica = seq_critica_atual

```

```

        else:

```

```

            seq_critica_atual = 0 # Quebra a sequência consecutiva

```

```

    return contagem_estados, maior_seq_critica

```

```

# --- FUNÇÃO DE SAÍDA DE DADOS (APRESENTAÇÃO) ---

```

```

def gerar_relatorio_diagnostico(nominal, qtd_validas, qtd_descartadas, estatisticas, estados,
seq_critica):

```

```
"""
```

7. Exibe o relatório formatado na tela (Camada de Apresentação / IO).

```
"""
```

```
print("=" * 55)
print("      RELATÓRIO DE DIAGNÓSTICO DA REDE ELÉTRICA")
print("=" * 55)
print(f"Tensão Nominal de Referência: {nominal} V")
print(f"Limiar de Alerta:          > {nominal * 0.10:.1f}V de desvio (10%)")
print(f"Limiar Crítico:          > {nominal * 0.20:.1f}V de desvio (20%)")
print("-" * 55)
print(f"Amostras Válidas:          {qtd_validas}")
print(f"Amostras Descartadas:       {qtd_descartadas}")
print("-" * 55)

if qtd_validas > 0:
    print("ESTATÍSTICAS DA TENSÃO:")
    print(f"  Mínima:          {estatisticas['minimo']:.2f} V")
    print(f"  Máxima:          {estatisticas['maximo']:.2f} V")
    print(f"  Média:           {estatisticas['media']:.2f} V")
    print(f"  Desvio Padrão Amostral: {estatisticas['desvio_padrao']:.2f} V")
    print("-" * 55)
    print("DISTRIBUIÇÃO DOS ESTADOS:")
    print(f"  Normal:          {estados['Normal']} amostra(s)")
    print(f"  Alerta [Desvio > 10%]: {estados['Alerta']} amostra(s)")
    print(f"  Crítico [Desvio > 20%]: {estados['Crítico']} amostra(s)")
    print("-" * 55)
    print("DETECÇÃO DE EVENTOS CRÍTICOS:")
    print(f"  Total de medições críticas: {estados['Crítico']}")
    print(f"  Maior sequência crítica consecutiva: {seq_critica}")
else:
    print("ERRO: Dados insuficientes para gerar estatísticas.")

print("=" * 55)
```

--- FUNÇÃO PRINCIPAL (ORQUESTRADORA) ---

```
def main():
```

```
    """
```

Função principal que coordena o fluxo de execução do programa.

Nenhuma lógica de cálculo é feita aqui, apenas o direcionamento dos dados.

```
    """
```

Configurações locais (evitando variáveis globais soltas)

TENSAO_NOMINAL = 220.0

LIMIAR_ALERTA_PCT = 0.10 # 10%

LIMIAR_CRITICO_PCT = 0.20 # 20%

Massa de testes (dados brutos contendo erros comuns de sensores)

```

dados_brutos = [
    215, 220, "erro_leitura", 218, 250, 270, 280, 266, 150, 600, 220, "NaN", 221
]

# Execução do fluxo através dos módulos
validas, descartadas = validar_e_filtrar_medicoes(dados_brutos, faixa_min=0.0,
faixa_max=500.0)

estats = calcular_estatisticas(validas)

estados, maior_seq = analisar_estados_e_sequencias(
    validas,
    nominal=TENSAO_NOMINAL,
    limiar_alerta=LIMIAR_ALERTA_PCT,
    limiar_critico=LIMIAR_CRITICO_PCT
)

# Exibição do resultado
gerar_relatorio_diagnostico(
    nominal=TENSAO_NOMINAL,
    qtd_validas=len(validas),
    qtd_descartadas=descartadas,
    estatisticas=estats,
    estados=estados,
    seq_critica=maior_seq
)

if __name__ == "__main__":
    main()

```